JOB AGENT

Technical Documentation

Full-Stack Job Application Automation Platform

Project Summary

Job Agent is a full-stack automation platform for discovering, scoring, and submitting job applications with a controllable approval flow. It combines a FastAPI backend, a React + Vite frontend, Playwright browser automation, and Cloudflare Workers AI to deliver an end-to-end pipeline from goal input to confirmed application.

1. Overview

Job Agent automates the full job-hunting workflow. Starting from a plain-language goal, the system searches LinkedIn, scores matches against a candidate profile, generates tailored cover letters, and submits Easy Apply applications — all with observable progress and persistent outcome tracking.

1.1 Who This Is For

- Developers who want to automate job discovery and ranking from a natural-language goal
- Teams who need a browser-based pipeline with observable progress and artifacts
- Anyone who wants persistent records of application outcomes and statuses
- Users who need a clear separation between dry-run validation and real submissions

1.2 Execution Modes

Mode	Description
Web (recommended)	React frontend with Firebase sign-in. Starts runs via API, polls progress, and renders step-level output and final reports.
CLI	Direct backend execution without the frontend. Suitable for scripted or headless environments.

1.3 Pipeline Stages

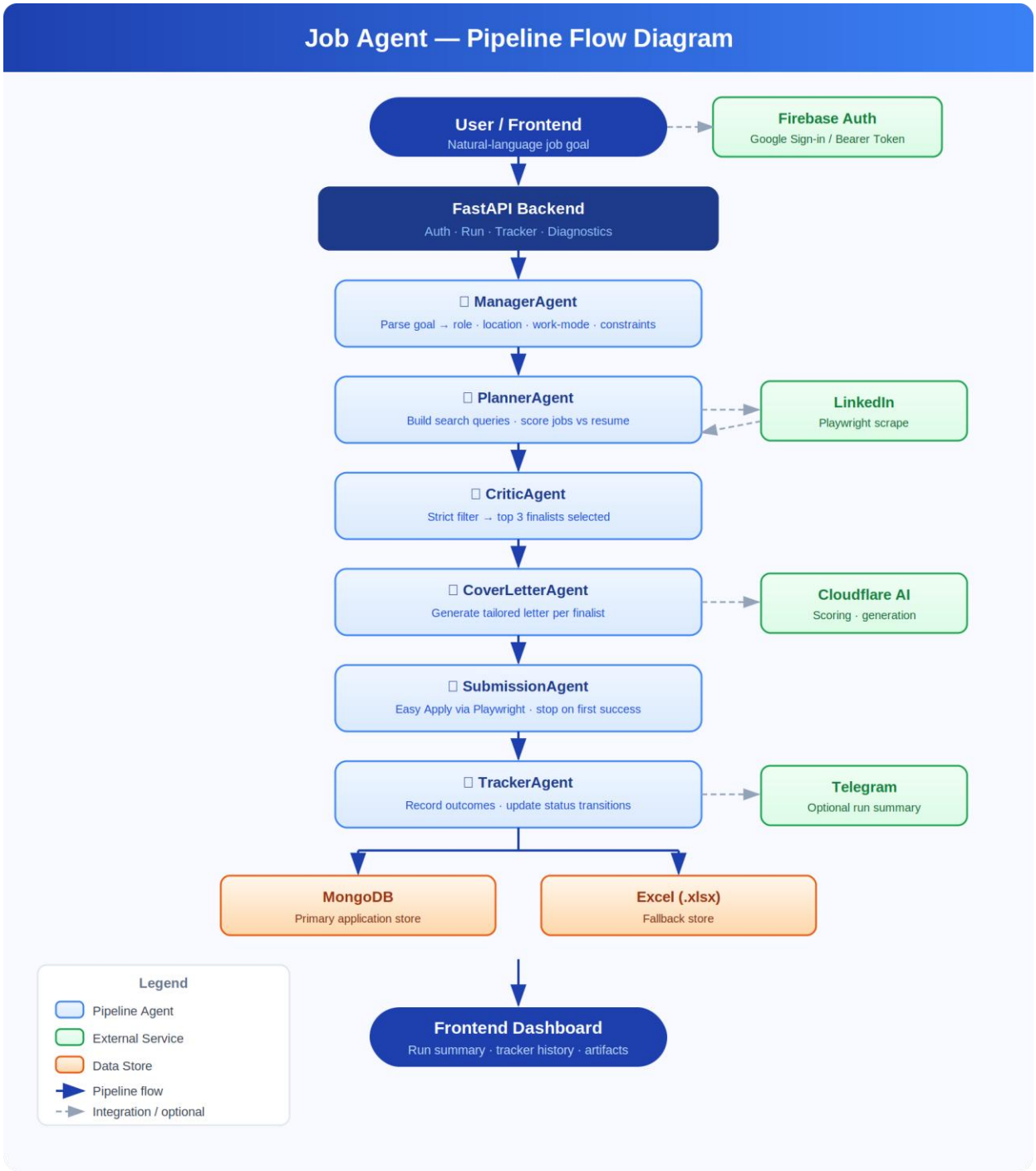
#	Stage	Description
1	Parse goal	Extract role, location, work-mode, and constraints from user input
2	Generate search plans	Build LinkedIn query strategies from parsed goal
3	Scrape jobs	Collect candidate job listings from LinkedIn via Playwright
4	Score jobs	Rank jobs against candidate profile and resume
5	Select finalists	Critic pass keeps the best 3 candidates
6	Generate cover letters	Produce tailored letters per finalist
7	Submit applications	Execute Easy Apply flow via Playwright
8	Store outcomes	Write statuses and results to MongoDB or Excel

1.4 Default Tuning

- Score top 5 jobs from scrape results
- Keep up to 3 critic-approved finalists
- Submit finalists in ranked order
- Stop after the first successful application

2. Pipeline Flow Diagram

The diagram below illustrates the complete end-to-end flow, from user sign-in through each agent stage to outcome storage and the frontend dashboard.



3. System Architecture

3.1 Component Overview

Layer	Responsibilities
Presentation (Frontend)	Collects credentials and preferences; starts runs; polls progress; renders step-level output, run summary, and tracker history
API (FastAPI)	Validates schemas; verifies Firebase token; provides health/startup diagnostics; exposes run, tracker, and Telegram interfaces
Orchestration (Service + Agents)	Enforces pipeline order and stop conditions; executes dry-run or real-run strategy; passes structured outputs between agents
Integration / Tool Layer	Playwright automation; AI model calls; tracker persistence; Telegram notifications

3.2 Technology Stack

Component	Technology
Backend framework	FastAPI (Python 3.11+)
Frontend	React + Vite (TypeScript, Node.js 20+)
Browser automation	Playwright (Chromium)
AI / ML	Cloudflare Workers AI
Authentication	Firebase Authentication (Google sign-in)
Primary data store	MongoDB Atlas or local MongoDB
Fallback data store	Excel (.xlsx)
Notifications	Telegram Bot API (optional)

3.3 Runtime Sequence (Web Mode)

- User signs in via Firebase Google provider in the React frontend
- Frontend sends Firebase ID token to POST /api/v1/auth/firebase-login
- Frontend submits run parameters to POST /api/v1/run/async
- FastAPI service creates a run_id and initialises run state
- ManagerAgent parses goal and constraints
- PlannerAgent builds queries and scores scraped jobs
- LinkedIn scraper collects candidate jobs via Playwright
- CriticAgent selects the best finalists
- CoverLetterAgent generates tailored cover letter content
- SubmissionAgent executes Easy Apply attempts

- TrackerAgent writes statuses and results
- Frontend polls GET /api/v1/run/{run_id} and renders progress and the final report

3.4 Reliability Strategy

- Shared run-scoped Playwright browser profile for scraper-to-submission handoff
- Async run state endpoint for frontend polling and recovery after disconnection
- MongoDB-first tracking with automatic Excel fallback
- Telegram failures do not affect the core pipeline result
- Diagnostics endpoint at /api/v1/startup for startup dependency visibility

4. Agent Pipeline

4.1 ManagerAgent

Entry point for every run. Parses the free-text job goal into structured fields: role, location, work mode, and any additional constraints. Normalises profile preferences for downstream agents.

4.2 PlannerAgent

Consumes the structured goal from ManagerAgent. Creates search plans and query strings optimised for LinkedIn. Scores each scraped job against the candidate resume and profile signal. Passes the top-ranked candidates to CriticAgent for final selection.

4.3 CriticAgent

Applies a stricter quality filter over the PlannerAgent results. Produces a small finalist set (up to 3 by default) that is forwarded to cover letter generation and submission.

4.4 CoverLetterAgent

Generates a customised cover letter for each finalist, combining the candidate resume with job-specific context. Output is stored as a runtime artifact and passed to the submission step.

4.5 SubmissionAgent

Drives the Playwright browser session to complete Easy Apply flows on LinkedIn. Attempts finalist jobs in ranked order and stops once the configured success condition is met (default: first successful submission).

4.6 TrackerAgent

Records each job attempt and every status transition throughout the run. Powers the history and stats endpoints used by the dashboard and audit exports.

5. API Reference

All endpoints are prefixed with /api/v1. Protected endpoints require a valid Firebase Bearer token.

5.1 Diagnostics

Endpoint	Method	Description
/api/v1/health	GET	Basic liveness check
/api/v1/startup	GET	Reports status of all startup dependencies
/api/v1/config	GET	Returns current non-sensitive configuration values

5.2 Authentication

Endpoint	Method	Description
/api/v1/auth/firebase-login	POST	Exchanges Firebase ID token for a session
/api/v1/me	GET	Returns the authenticated user's profile (protected)

5.3 Pipeline

Endpoint	Method	Description
/api/v1/run	POST	Start a synchronous pipeline run (protected)
/api/v1/run/async	POST	Start an asynchronous run and return a run_id (protected)
/api/v1/run/{run_id}	GET	Poll run state and final report (protected)

5.4 Tracker

Endpoint	Method	Description
/api/v1/tracker/stats	GET	Aggregate application statistics (protected)
/api/v1/tracker/history	GET	Full application history with statuses (protected)
/api/v1/tracker/update-status	POST	Manually update a tracked status (protected)

5.5 Telegram & Frontend Support

Endpoint	Method	Description
/api/v1/telegram/status	GET	Check Telegram bot connectivity
/api/v1/telegram/test	POST	Send a test notification
/api/v1/features	GET	Feature flags for the frontend
/api/v1/docs-summary	GET	Returns a summary for the onboarding docs panel

6. Authentication Model

#	Step
1	User signs in via the Firebase Google provider on the React frontend
2	Firebase returns a short-lived ID token to the frontend
3	Frontend includes the token as Authorization: Bearer <token> on every API request
4	FastAPI backend verifies the token using Firebase Admin SDK and serviceAccountKey.json
5	Protected endpoints execute only for requests with a valid, non-expired identity

7. Repository Layout

Path	Purpose
backend/agent/	Agent modules: manager, planner, critic, submission, tracker
backend/api/	FastAPI app, service orchestration, request/response models
backend/tools/	Cover letter, resume, job, submission, tracking, Telegram, JSONL tools
backend/web_scrapping/	LinkedIn Playwright scraper
backend/tests/	Backend unit and integration tests
backend/data/	Runtime artifacts: logs, XLSX, JSONL, browser profiles
backend/config.py	Central configuration and environment variable loading
backend/main.py	CLI entry point
frontend/src/components/	React UI components (dashboard, config, output, overview)
frontend/src/services/api.ts	Typed API client for the FastAPI backend
frontend/src/config/firebase.ts	Firebase SDK initialisation
SETUP_GUIDE.md	Step-by-step environment and credential setup walkthrough
COMMANDS.md	Command cookbook for daily operation

8. Setup & Quick Start

8.1 Requirements

Dependency	Version	Notes
Python	3.11+	Required
Node.js	20+	Required
npm	10+	Required
Firebase project	—	Required for Google sign-in
LinkedIn account	—	Required for scraping and submission
MongoDB Atlas / local	—	Recommended; Excel used as fallback

Telegram bot	—	Optional; for run summary notifications

8.2 Environment Setup

```
python -m venv .venv
.\\. venv\\Scripts\\Activate.ps1
pip install -r requirements.txt
python -m playwright install chromium
cd frontend && npm install && cd ..
```

8.3 Start the Backend

```
cd backend
uvicorn api.app:app --host 0.0.0.0 --port 8000 --reload
```

8.4 Start the Frontend

```
cd frontend && npm run dev
```

Then open http://127.0.0.1:5173 in your browser.

8.5 Recommended First-Run Workflow

1. Start the backend and confirm /api/v1/startup shows all services healthy
2. Start the frontend and sign in with your Google account
3. Run a dry-run first (DRY_RUN=true) to validate scoring and finalist selection
4. Inspect logs, scoring decisions, and finalists in backend/data/
5. Enable live submission only after dry-run confidence is established
6. Review tracker history and exported artifacts for audit

9. Runtime Artifacts

Artifact	Purpose
agent.log	Full agent execution log for debugging
list.xlsx	Excel export of all tracked applications
agent_context.jsonl	Structured agent context for the most recent run
agent_context_<run_id>.jsonl	Per-run context archive
runtime_browser_profiles/	Playwright browser profiles scoped per run
runtime_resumes/	Uploaded or parsed resume files
debug/	Additional debug snapshots and screenshots

10. Testing

10.1 Backend Tests

```
cd backend && pytest -q
pytest tests/test_manager_agent.py -q
pytest tests/test_planner_agent.py -q
pytest tests/test_tracker_agent.py -q
pytest tests/test_integration_dry_run.py -q
```

10.2 Frontend Checks

```
cd frontend && npm run lint && npm run build && npx tsc -b
```

11. Troubleshooting

Symptom	Likely Cause / Resolution
'Failed to fetch' in frontend	Backend not running, wrong VITE_API_BASE_URL, or CORS origin mismatch
/api/v1/startup shows Mongo ok=False	Check MONGODB_URI, credentials, Atlas network allowlist, and cluster availability
Firebase login fails	Verify frontend Firebase env vars, backend serviceAccountKey.json, and local system clock skew
LinkedIn opens feed instead of job page	Session restoration issue — confirm logs show a recovered /jobs/view/... URL
Telegram not sending after run	TELEGRAM_ENABLED is likely set to false

12. Security Notes

- Never commit backend/.env — it contains secrets including LinkedIn credentials and API keys
- Never commit frontend/.env — it contains Firebase configuration
- Treat backend/serviceAccountKey.json as a highly sensitive credential; add it to .gitignore
- Do not expose or log LinkedIn credentials anywhere in the codebase
- Rotate any leaked keys, tokens, or secrets immediately